

รายงานการพัฒนาและประเมินแบบจำลองถดถอยผสม

การจัดกลุ่ม K-Means และการถดถอยพหุนาม OLS เฉพาะกลุ่ม

รายงานฉบับสมบูรณ์พร้อมผลการทดลองและโค้ด Python

ผู้จัดทำ

นางสาวซารีนา แยนา 6720310005

นายอนัส ยะโยะ 6720310020

นางสาวฟิรดาว เจ๊ะเล๊ะ 6720310111

นายอับดุลเลาะห์ สาหาด 6720310127

เสนอ

อนุรักษ์ บูสะมัญ

รายวิชา

746-345 PROGRAMMING NUMERICAL METHODS

ภาคเรียนที่ 1 ปีการศึกษา 2569

บทคัดย่อ

รายงานนี้พัฒนาและประเมินแบบจำลองถดถอยแบบผสม โดยใช้ K-Means แบ่งข้อมูลตามตัวแปรอิสระ x แล้วประมาณสมการพหุนามด้วยวิธีกำลังสองน้อยสุด (Ordinary Least Squares: OLS) แยกตามกลุ่ม เปรียบเทียบกับแบบจำลองพหุนามกำลังสอง OLS ที่ fit จากข้อมูลฝึกทั้งหมดเพียงแบบจำลองเดียว การทดลองใช้ข้อมูลสังเคราะห์ 450 ตัวอย่างซึ่งสร้างจากความสัมพันธ์กำลังสองสามช่วงและสัญญาณรบกวนแบบปกติ แบ่งเป็นข้อมูลฝึก 315 ตัวอย่างและข้อมูลทดสอบ 135 ตัวอย่างด้วย random_state=42 โดยกำหนดจำนวนกลุ่ม k=3 และดีกรีพหุนามเท่ากับ 2

บนชุดทดสอบ แบบจำลอง K-Means ร่วมกับ Local OLS ได้ MSE 0.1847, RMSE 0.4298 และ R^2 0.9739 ส่วน Global OLS ได้ MSE 2.5298, RMSE 1.5905 และ R^2 0.6429 คิดเป็น MSE ที่ลดลง 92.70% เมื่อเทียบกับ Global OLS ผลนี้ชี้ว่าแบบจำลองรายกลุ่มจับรูปทรงข้อมูลที่เปลี่ยนตามช่วงได้ดีกว่าในตัวอย่างนี้ อย่างไรก็ตาม ผลมาจากข้อมูลสังเคราะห์และการแบ่งชุดทดสอบครั้งเดียว จึงยังไม่ยืนยันความสามารถในการนำไปใช้ทั่วไปกับข้อมูลจริง ควรเลือก k และดีกรีด้วย cross-validation บนชุดฝึก แล้วประเมินชุดทดสอบที่กันไว้เพียงครั้งสุดท้าย

คำสำคัญ: K-Means, การถดถอยพหุนาม, OLS, การถดถอยเฉพาะกลุ่ม, MSE

สารบัญ

เรื่อง **หน้า**

บทคัดย่อ 2

บทที่ 1 บทนำ 4

บทที่ 2 ทฤษฎีและกรอบแนวคิด 5

บทที่ 3 วิธีดำเนินงาน 6

บทที่ 4 ผลการทดลอง 8

บทที่ 5 อภิปรายผลและข้อจำกัด 10

บทที่ 6 สรุปและข้อเสนอแนะ 11

เอกสารอ้างอิง 12

ภาคผนวก ก โค้ด Python ฉบับสมบูรณ์ 13

สารบัญตาราง

ตารางที่ 1 พารามิเตอร์และสภาพแวดล้อมการทดลอง 6

ตารางที่ 2 ผลการเปรียบเทียบแบบจำลองบนชุดทดสอบ 8

ตารางที่ 3 ค่าสัมประสิทธิ์ OLS รายกลุ่ม 8

ตารางที่ 4 ข้อดีและข้อจำกัดของแต่ละวิธี 10

สารบัญภาพ

ภาพที่ 1 เส้นแนวโน้มของแบบจำลองบนข้อมูลชุดฝึกและชุดทดสอบ 9

บทที่ 1 บทนำ

ความเป็นมาและความสำคัญ

การถดถอยเชิงเส้นเป็นเครื่องมือพื้นฐานที่อธิบายความสัมพันธ์ระหว่างตัวแปรได้ด้วยสัมประสิทธิ์
แต่ความสัมพันธ์ในงานจริงอาจเปลี่ยนไปตามช่วงหรือสภาวะ เช่น เซนเซอร์ที่ตอบสนองต่างกันเมื่ออุณหภูมิเปลี่ยน
หรือความต้องการสินค้าที่มีรูปแบบตามฤดูกาล การ fit สมการเดียวอาจไม่สามารถแทนทุกช่วงได้ดี
ขณะที่การเพิ่มดีกรีพหุนามให้สมการเดียวอาจทำให้สมการซับซ้อนและไวต่อข้อมูล

รายงานนี้ศึกษาวิธีแบ่งข้อมูลเป็นกลุ่มด้วย K-Means แล้ว fit OLS เฉพาะภายในแต่ละกลุ่ม
แนวทางดังกล่าวผสานการจัดกลุ่มแบบไม่กำกับกับการถดถอยแบบตีความได้ เพื่อให้สมการปรับตามส่วนต่าง ๆ ของข้อมูล [1–4]

ปัญหาที่ศึกษา

คำถามหลักคือ เมื่อข้อมูลมีแนวโน้มไม่เป็นเส้นตรงและเปลี่ยนตามช่วง การใช้ K-Means เลือกกลุ่มก่อนทำนายด้วย OLS
รายกลุ่มให้ความคลาดเคลื่อนต่ำกว่าการใช้ OLS พหุนามทั่วทั้งชุดข้อมูลหรือไม่ และวิธีนี้มีข้อแลกเปลี่ยนด้านการตีความ ความต่อเนื่อง
และปริมาณข้อมูลอย่างไร

วัตถุประสงค์

1. สร้างโปรแกรม Python ที่จัดกลุ่มข้อมูลด้วย K-Means และประมาณสมการพหุนามด้วย OLS แยกตามกลุ่ม
2. คำนวณค่าสัมประสิทธิ์ วาดกราฟเส้นแนวโน้ม และบันทึกผลลัพธ์เป็นไฟล์ที่ตรวจสอบได้
3. เปรียบเทียบ MSE, RMSE และ R^2 ระหว่างแบบจำลองผสมกับ Global Polynomial OLS บนชุดทดสอบเดียวกัน
4. สรุปจุดแข็ง ข้อจำกัด และเงื่อนไขที่ควรพิจารณาก่อนประยุกต์ใช้กับข้อมูลจริง

ขอบเขตและข้อจำกัดของการศึกษา

การสาธิตใช้ตัวแปรอิสระ x หนึ่งตัว ข้อมูลสังเคราะห์ 450 ตัวอย่าง จำนวนกลุ่ม 3 กลุ่ม และพหุนามดีกรี 2 แบ่งข้อมูล train/test 70/30
ด้วย seed เดียว ไม่ได้ทำการค้นหาค่า k หรือดีกรีที่เหมาะสมด้วย cross-validation และไม่ได้ทดสอบกับข้อมูลจากโดเมนจริง
ดังนั้นผลตัวเลขใช้เปรียบเทียบวิธีในชุดทดลองนี้เท่านั้น

ประโยชน์ที่คาดว่าจะได้รับ

ผู้อ่านจะเห็นลำดับการทำงานตั้งแต่สร้างข้อมูล จัดกลุ่ม fit OLS จนถึงประเมินผล และสามารถนำโค้ดไปปรับใช้เป็นต้นแบบได้
การแสดงสมการและผลทดสอบช่วยให้เห็นทั้งความสามารถในการจับรูปทรงและความเสี่ยงที่เกิดจากการแบ่งกลุ่ม

บทที่ 2 ทฤษฎีและกรอบแนวคิด

การจัดกลุ่มด้วย K-Means

K-Means กำหนดจำนวนกลุ่ม k แล้วหาจุดศูนย์กลางที่ทำให้ผลรวมระยะกำลังสองภายในกลุ่มต่ำ อัลกอริทึมกำหนดแต่ละตัวอย่างให้อยู่กับ centroid ที่ใกล้ที่สุดและปรับ centroid ซ้ำจนลู่เข้า [1, 2]

$$J = \text{sum_i min_j } ||x_i - mu_j||^2$$

จำนวนกลุ่มต้องกำหนดล่วงหน้า และผลขึ้นกับการเริ่มต้น centroid จึงมักใช้หลายการเริ่มต้น ในโปรแกรมนี้กำหนด n_clusters=3, n_init=20 และ random_state=42 K-Means ใช้เฉพาะ x เพื่อให้สามารถกำหนดกลุ่มของข้อมูลใหม่ได้โดยไม่ต้องทราบค่า y

พหุนามและวิธีกำลังสองน้อยสุด

PolynomialFeatures สร้างฟีเจอร์กำลังของ x เช่น x และ x^2 จากนั้น LinearRegression ประมาณสัมประสิทธิ์ด้วย OLS โดยลดผลรวมกำลังสองของ residual [3, 4] สำหรับดีกรี 2 สมการของกลุ่ม c เขียนได้ดังนี้

$$\text{yhat_c(x)} = b0_c + b1_c*x + b2_c*x\^2$$
$$\text{bhat_c} = \text{argmin_b sum_i } (y_i - b0 - b1*x_i - b2*x_i\^2)\^2$$

แบบจำลองผสมใช้ centroid ที่เรียนรู้จากชุดฝึกกำหนดกลุ่มของ x ใหม่ แล้วคำนวณค่าทำนายจากสมการกลุ่มนั้น ส่วน Global OLS ใช้สัมประสิทธิ์ชุดเดียวกับทุกตัวอย่าง

ตัวชี้วัดและเกณฑ์เปรียบเทียบ

MSE วัดค่าเฉลี่ยความคลาดเคลื่อนกำลังสอง RMSE แปลงค่ากลับเป็นหน่วยของ y และ R^2 เทียบ residual กับความแปรปรวนรอบค่าเฉลี่ย โดยทั่วไป MSE และ RMSE ยิ่งต่ำยิ่งดี ส่วน R^2 ยิ่งสูงยิ่งดี [5, 6]

$$\text{MSE} = \text{mean}((y_i - yhat_i)\^2) \quad \text{RMSE} = \text{sqrt(MSE)}$$
$$R2 = 1 - SSE/SST = 1 - \text{sum_i}(y_i\text{-}yhat_i)\^2 \,/\, \text{sum_i}(y_i\text{-}ybar)\^2$$

กรอบการทดลองจึงเปรียบเทียบสองขั้นตอน ได้แก่ (1) แบ่งข้อมูลและฝึกด้วยชุดฝึกเดียวกัน (2) ใช้แบบจำลองทั้งสองทำนายชุดทดสอบ แล้วเปรียบเทียบตัวชี้วัดและกราฟ ภาพรวมใช้ข้อมูล x สำหรับการจัดกลุ่มและใช้ y เป็นเป้าหมายในการถดถอย

บทที่ 3 วิธีดำเนินงาน

แบบแผนและชุดข้อมูล

สร้างข้อมูลสังเคราะห์ x จำนวน 450 จุดโดยสุ่มสม่ำเสมอในช่วง -6 ถึง 6 ความสัมพันธ์พื้นฐานของ y เป็นพาราโบลาสามช่วง และบวกสัญญาณรบกวนปกติที่มีส่วนเบี่ยงเบนมาตรฐาน 0.42 สมการกำหนดไว้ในโปรแกรมภาคผนวกเพื่อให้ทำซ้ำได้ด้วย random_state=42

$$y_i = f(x_i) + e_i,\ e_i \sim \text{Normal}(0, 0.42^2)$$
$$f(x) = 0.55*(x + 3.2)^2 - 3.6, \text{ if } x < -2$$
$$0.35*x^2 + 0.65*x - 0.8, \text{ if } -2 <= x < 2$$
$$-0.48*(x - 3.4)^2 + 4.0, \text{ if } x >= 2$$

ใช้ train_test_split แบ่ง 70% เป็นชุดฝึกและ 30% เป็นชุดทดสอบ โดยกำหนด random_state=42 ข้อมูล 315 แถวใช้ fit K-Means และ OLS ส่วน 135 แถวใช้คำนวณผลทำนายครั้งสุดท้าย การ fit ทุกขั้นตอนทำจากชุดฝึก

พารามิเตอร์และสภาพแวดล้อม

ตารางที่ 1 สรุปค่าที่กำหนดในโปรแกรมและสภาพแวดล้อมที่ใช้สร้างผลรายงาน

รายการ	ค่าที่ใช้
จำนวนตัวอย่าง	450
การแบ่งข้อมูล	ฝึก 315 (70%) ทดสอบ 135 (30%)
K-Means	k=3, n_init=20, random_state=42
ตัวแปรจัดกลุ่ม	x เท่านั้น
แบบจำลอง OLS	PolynomialFeatures ดีกรี 2 และ LinearRegression
สภาพแวดล้อมที่รัน	Python 3.14, NumPy 2.5.2, pandas 3.0.6, scikit-learn 1.9.1, matplotlib 3.11.2

ขั้นตอนการพัฒนาและประเมิน

1. สร้างข้อมูลและแบ่ง train/test ด้วย seed คงที่
2. fit K-Means จาก x ในชุดฝึก แล้วนำ label ไป fit OLS พหุนามแยกสำหรับแต่ละกลุ่ม
3. fit Global OLS จากชุดฝึกทั้งหมดโดยใช้ดีกรีพหุนามเดียวกัน
4. ทำนายชุดทดสอบด้วยแบบจำลองทั้งสอง คำนวณ MSE, RMSE, R^2 และบันทึกกราฟกับตาราง

การใช้ AI และการตรวจผล

AI ช่วยร่างโครงสร้างโค้ดสำหรับสร้างข้อมูล จัดกลุ่ม fit OLS คำนวณตัวชี้วัด และวาดกราฟ จากนั้นรันโปรแกรมจริง ตรวจค่าสัมประสิทธิ์และคะแนนที่ได้ และตรึง seed เพื่อทำซ้ำได้ AI ช่วยการเขียนโค้ด แต่ไม่ได้แทนการตรวจสอบวิธีแบ่งข้อมูล สมมติฐาน และความเหมาะสมของผลกับงานจริง

บทที่ 4 ผลการทดลอง

ผลการเปรียบเทียบ

ทดสอบกับข้อมูล 135 ตัวอย่างซึ่งไม่ได้ใช้ fit แบบจำลอง ตารางที่ 2 แสดงค่าที่ได้จากการรันโปรแกรม ค่า MSE และ RMSE ต่ำลง และ R^2 สูงขึ้นสำหรับแบบจำลอง K-Means ร่วมกับ Local OLS

ตารางที่ 2 ผลการเปรียบเทียบแบบจำลองบนชุดทดสอบ

แบบจำลอง	MSE	RMSE	R^2	MSE ลดลง
K-Means และ Local OLS	0.1847	0.4298	0.9739	92.70%
Global Polynomial OLS	2.5298	1.5905	0.6429	0.00%

MSE ของแบบจำลองผสมต่ำกว่า Global OLS 2.3451 หน่วย² หรือ 92.70% ตามสูตร (1 − MSEผสม/MSEglobal) × 100 ส่วน R^2 ต่างกัน 0.3310 บนชุดทดสอบนี้ ผลดังกล่าวเข้ากับข้อมูลที่สร้างจากแนวโน้มคนละช่วง
แต่การแบ่งข้อมูลเพียงครั้งเดียวยังไม่บอกความแปรปรวนของคะแนนระหว่าง fold

สัมประสิทธิ์ของสมการเฉพาะกลุ่ม

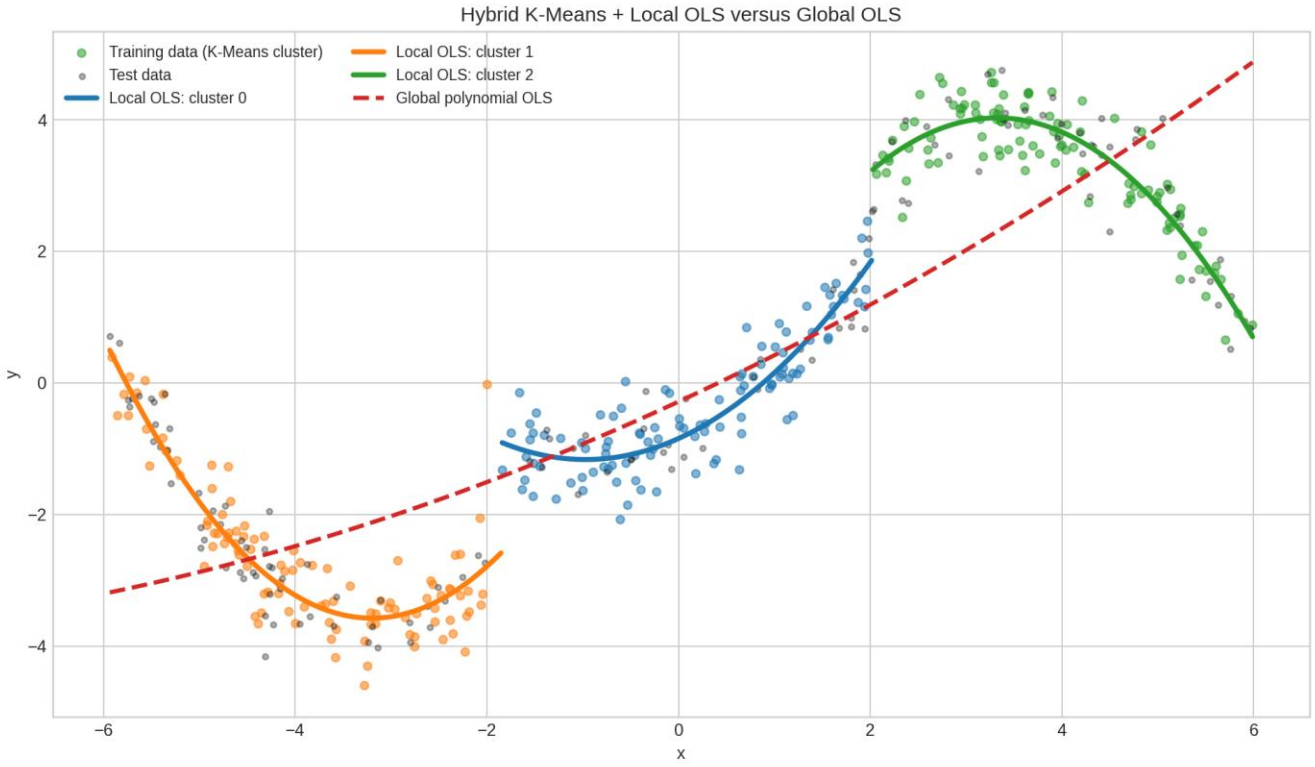
ตารางที่ 3 เรียง label ตามตำแหน่ง centroid จาก x ต่ำไป x สูง หมายเลขกลุ่มเป็น label ที่ K-Means สร้างและอาจเปลี่ยนได้เมื่อฝึกใหม่ แม้จะเป็นกลุ่มเดียวกันในเชิงตำแหน่ง

กลุ่ม	ศูนย์กลาง x	จำนวนฝึก	β_0	β_1	β_2
1	-3.8389	102	2.0085	3.4896	0.5451
0	0.1380	106	-0.8457	0.6594	0.3395
2	3.9136	107	-1.1371	3.1088	-0.4678

Global OLS ให้สัมประสิทธิ์ β_0 = -0.2857, β_1 = 0.6741 และ β_2 = 0.0312
ตารางนี้แสดงค่าสัมประสิทธิ์รายกลุ่มให้ตรวจสอบแยกจากคะแนนรวม โดยสมการทำนายของแต่ละกลุ่มมีรูปแบบตามสมการพหุนามในบทที่ 2

กราฟผลการทำนาย

ภาพที่ 1 แสดงเส้น Local OLS ของแต่ละกลุ่มเทียบกับเส้น Global OLS จุดข้อมูลฝึกใช้สีตามกลุ่ม จุดชุดทดสอบเป็นสีดำแบบโปร่งแสง (มองเห็นเป็นสีเทา) เส้นแต่ละกลุ่มอาจไม่ต่อเนื่องตรงขอบเขต เพราะ fit สมการแยกกัน



ภาพที่ 1 เส้นแนวโน้มของแบบจำลองบนข้อมูลชุดฝึกและชุดทดสอบ

บทที่ 5 อภิปรายผลและข้อจำกัด

การตีความผล

คะแนนที่ดีขึ้นสอดคล้องกับรูปแบบข้อมูลสังเคราะห์ซึ่งเปลี่ยนความโค้งตามช่วง สมการ Global กำลังสองหนึ่งสมการมีข้อจำกัดในการติดตามพฤติกรรมทั้งสามช่วง ขณะที่ Local OLS มีสัมประสิทธิ์แยกกัน จึงปรับความชันและความโค้งได้ตามตำแหน่งข้อมูล อย่างไรก็ตาม การจัดกลุ่มจาก x เพียงตัวเดียวในตัวอย่างนี้แบ่งโดเมนเป็นช่วงตามตำแหน่ง ไม่ได้พิสูจน์ว่าการจัดกลุ่ม K-Means จะค้นพบระบอบข้อมูลที่มีความหมายในทุกงาน

ข้อดีและข้อจำกัดของแต่ละวิธี

ตารางที่ 4 เปรียบเทียบคุณสมบัติที่มีผลต่อการเลือกใช้

ประเด็น	K-Means และ Local OLS	Global OLS
รูปทรงข้อมูล	ปรับสมการตามแนวโน้มแต่ละบริเวณ	อาจ underfit เมื่อรูปแบบเปลี่ยนหลายช่วง
การตีความ	อธิบายสมการรายกลุ่มได้ แต่ต้องอธิบายกฎแบ่งกลุ่มเพิ่ม	สื่อสารง่ายเพราะมีสมการเดียว
ข้อมูลและความเสี่ยง	ต้องมีข้อมูลพอในทุกกลุ่มและเสี่ยง overfit เมื่อเพิ่ม k หรือดีกรี	ใช้ข้อมูลรวมและซับซ้อนน้อยกว่า แต่อาจแทนโครงสร้างไม่พอ
เส้นที่ขอบเขต	ค่าทำนายอาจกระโดดเมื่อเปลี่ยนกลุ่ม	พหุนามต่อเนื่องตลอดช่วง

ข้อจำกัดของผลทดลอง

การประเมินใช้ข้อมูลสังเคราะห์ชุดเดียวและ random holdout ครั้งเดียว ไม่มีการรายงานค่าเฉลี่ยหรือส่วนเบี่ยงเบนของคะแนนจากหลาย fold จึงไม่ควรสรุปว่า Hybrid จะชนะ Global ในข้อมูลจริงทุกกรณี นอกจากนี้จำนวนกลุ่มและดีกรีถูกกำหนดไว้ล่วงหน้า ไม่ได้เลือกจากกระบวนการปรับเทียบ ในงานจริงอาจใช้ silhouette analysis ช่วยพิจารณาจำนวนกลุ่มที่เหมาะสมร่วมกับ cross-validation [7]

K-Means ใช้ระยะทางแบบ Euclidean และทำงานได้เหมาะเมื่อโครงสร้างกลุ่มสอดคล้องกับสมมติฐานของ centroid หากใช้หลายฟีเจอร์ที่มีหน่วยต่างกัน ต้อง fit scaler จากชุดฝึกและแปลงข้อมูลใหม่ก่อนจัดกลุ่ม การแยก OLS ยังมีข้อควรตรวจเรื่อง residual, outlier และความสัมพันธ์สูงระหว่างฟีเจอร์พหุนาม [2, 4]

บทที่ 6 สรุปและข้อเสนอแนะ

สรุปผล

โปรแกรมที่พัฒนาสามารถสร้างข้อมูลสังเคราะห์ จัดกลุ่มด้วย K-Means สร้าง OLS พหุนามรายกลุ่มและแบบ Global คำนวณตัวชี้วัด วาดกราฟ และบันทึก CSV/JSON ได้ครบ ผลทดสอบ 135 ตัวอย่างให้ Hybrid MSE 0.1847 และ R^2 0.9739 เทียบกับ Global MSE 2.5298 และ R^2 0.6429 จึงเห็นความได้เปรียบของแบบจำลองย่อยในข้อมูลทดลองนี้

ค่าที่ได้เป็นผลสาธิตจากข้อมูลที่สร้างขึ้นและ train/test split เดียว
รายงานจึงสรุปได้เฉพาะว่ากระบวนการทำงานและผลเปรียบเทียบเป็นไปตามที่คาดในชุดข้อมูลนี้ ไม่ใช่ข้อสรุปเชิงทั่วไปสำหรับข้อมูลจริง

ข้อเสนอแนะสำหรับการนำไปใช้

1. เริ่มจากตรวจกราฟและความรู้โดเมนว่ามีช่วงหรือสภาวะที่ความสัมพันธ์เปลี่ยนจริงหรือไม่ หากไม่มีเหตุผลรองรับ การเพิ่มกลุ่มอาจเพิ่มความซับซ้อนโดยไม่ช่วยการทำนาย
2. เปรียบเทียบ k และ degree ด้วย cross-validation บนชุดฝึก เลือกโดยดู validation MSE และความสมเหตุสมผลของขนาดกลุ่ม พร้อมล็อก test set ไว้ประเมินหลังเลือกแบบจำลอง
3. ใช้ pipeline สำหรับ scaling และ preprocessing ในกรณีหลายตัวแปร เพื่อป้องกันการรั่วไหลของข้อมูลและให้ระยะทางของ K-Means ไม่ถูกครอบงำโดยฟีเจอร์ที่มีหน่วยใหญ่
4. ตรวจความต่อเนื่องที่รอยต่อ ความคลาดเคลื่อนรายกลุ่ม และ residual หากจำเป็นต้องได้เส้นเรียบ ให้เปรียบเทียบกับ spline หรือ mixture of experts

ไฟล์ที่โปรแกรมสร้าง

- comparison_plot.png กราฟเส้นแนวโน้มและจุดข้อมูล
- metrics.csv และ results.json คะแนนแบบจำลองและสัมประสิทธิ์ Global
- local_coefficients.csv สัมประสิทธิ์รายกลุ่ม และ dataset.csv ข้อมูลสังเคราะห์

เอกสารอ้างอิง

[1] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Vol. 1, pp. 281–297. University of California Press. https://ndlsearch.ndl.go.jp/en/books/R100000136-I1570854176026181760

[2] scikit-learn developers. KMeans API reference, scikit-learn 1.9.1. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html

[3] scikit-learn developers. PolynomialFeatures API reference, scikit-learn 1.9.1. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html

[4] scikit-learn developers. LinearRegression API reference, scikit-learn 1.9.1. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

[5] scikit-learn developers. mean_squared_error API reference, scikit-learn 1.9.1. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_squared_error.html

[6] scikit-learn developers. r2_score API reference, scikit-learn 1.9.1. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2_score.html

[7] scikit-learn developers. Selecting the number of clusters with silhouette analysis on KMeans clustering. https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html

สืบค้นเอกสารออนไลน์เมื่อ 19 ตุลาคม 2569

ภาคผนวก ก โค้ด Python ฉบับสมบูรณ์

โค้ดด้านล่างสร้างข้อมูลสังเคราะห์ ฝึกแบบจำลองทั้งสองวิธี คำนวณค่าสัมประสิทธิ์และตัวชี้วัด แล้วบันทึกกราฟกับตารางผลลัพธ์ไว้ในโฟลเดอร์ outputs

```
"""Hybrid K-Means + local polynomial OLS regression demonstration.

Run:
    python hybrid_kmeans_ols.py

Outputs are written to ./outputs.  The data set is synthetic so the example is
fully reproducible; replace make_dataset() with pd.read_csv(...) for real data.
"""

from pathlib import Path
import json
import os

# Keep Matplotlib's cache inside the project, including on restricted systems.
PROJECT_DIR = Path(__file__).resolve().parent
os.environ.setdefault("MPLCONFIGDIR", str(PROJECT_DIR / ".matplotlib"))
os.environ.setdefault("MPLBACKEND", "Agg")  # non-interactive, portable image output
os.environ.setdefault("LOKY_MAX_CPU_COUNT", "1")  # avoids platform core-detection warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

RANDOM_STATE = 42
N_CLUSTERS = 3
POLYNOMIAL_DEGREE = 2
OUT_DIR = PROJECT_DIR / "outputs"


def make_dataset(n_samples=450, random_state=RANDOM_STATE):
```

```
    """Make a noisy, piecewise non-linear 1D data set."""
    rng = np.random.default_rng(random_state)
    x = np.sort(rng.uniform(-6, 6, n_samples))
    y = np.piecewise(
        x,
        [x < -2, (x >= -2) & (x < 2), x >= 2],
        [
            lambda z: 0.55 * (z + 3.2) ** 2 - 3.6,
            lambda z: 0.35 * z**2 + 0.65 * z - 0.8,
            lambda z: -0.48 * (z - 3.4) ** 2 + 4.0,
        ],
    )
    y = y + rng.normal(0, 0.42, n_samples)
    return pd.DataFrame({"x": x, "y": y})

def build_ols(degree=POLYNOMIAL_DEGREE):
    """Polynomial basis followed by OLS (sklearn LinearRegression)."""
    return Pipeline([
        ("poly", PolynomialFeatures(degree=degree, include_bias=False)),
        ("ols", LinearRegression()),
    ])

def predict_hybrid(x, kmeans, local_models):
    """Assign each x to its K-Means cluster and predict with that cluster's OLS."""
    labels = kmeans.predict(x.reshape(-1, 1))
    prediction = np.empty(len(x))
    for label, model in local_models.items():
        mask = labels == label
        prediction[mask] = model.predict(x[mask].reshape(-1, 1))
    return prediction, labels

def polynomial_equation(model):
    """Return readable coefficients: intercept + b1*x + b2*x^2 + ..."""
    ols = model.named_steps["ols"]
    return {"intercept": float(ols.intercept_),
            **{f"x^{i + 1}": float(value) for i, value in enumerate(ols.coef_)}}
```

```
def main():
    OUT_DIR.mkdir(exist_ok=True)
    data = make_dataset()
    train, test = train_test_split(data, test_size=0.30, random_state=RANDOM_STATE)
    x_train = train[["x"]].to_numpy()
    x_test = test[["x"]].to_numpy()
    y_train = train["y"].to_numpy()
    y_test = test["y"].to_numpy()

    # Cluster using the predictor X only, avoiding target leakage at prediction time.
    kmeans = KMeans(n_clusters=N_CLUSTERS, n_init=20, random_state=RANDOM_STATE)
    train_labels = kmeans.fit_predict(x_train)

    local_models = {}
    local_rows = []
    for label in range(N_CLUSTERS):
        mask = train_labels == label
        model = build_ols().fit(x_train[mask], y_train[mask])
        local_models[label] = model
        local_rows.append({
            "cluster": label,
            "center_x": float(kmeans.cluster_centers_[label, 0]),
            "train_samples": int(mask.sum()),
            **polynomial_equation(model),
        })

    global_model = build_ols().fit(x_train, y_train)
    hybrid_pred, test_labels = predict_hybrid(x_test.ravel(), kmeans, local_models)
    global_pred = global_model.predict(x_test)
    metrics = pd.DataFrame([
        {"model": "Hybrid K-Means + Local OLS", "MSE": mean_squared_error(y_test, hybrid_pred),
         "RMSE": mean_squared_error(y_test, hybrid_pred) ** 0.5, "R2": r2_score(y_test, hybrid_pred)},
        {"model": "Global polynomial OLS", "MSE": mean_squared_error(y_test, global_pred),
         "RMSE": mean_squared_error(y_test, global_pred) ** 0.5, "R2": r2_score(y_test, global_pred)},
    ])
    metrics["MSE_improvement_vs_global_pct"] = [
        (1 - metrics.loc[0, "MSE"] / metrics.loc[1, "MSE"]) * 100, 0.0
    ]
```

```
# Save complete tabular outputs.
data.to_csv(OUT_DIR / "dataset.csv", index=False)
pd.DataFrame(local_rows).sort_values("center_x").to_csv(OUT_DIR / "local_coefficients.csv", index=False)
metrics.to_csv(OUT_DIR / "metrics.csv", index=False)
with open(OUT_DIR / "results.json", "w", encoding="utf-8") as file:
    json.dump({"metrics": metrics.to_dict(orient="records"),
               "global_coefficients": polynomial_equation(global_model)}, file, indent=2)

# Plot local curves by predicted cluster across a dense x grid.
plt.style.use("seaborn-v0_8-whitegrid")
fig, ax = plt.subplots(figsize=(11, 6.5))
colors = ["#1f77b4", "#ff7f0e", "#2ca02c"]
x_grid = np.linspace(data.x.min(), data.x.max(), 1000)
grid_labels = kmeans.predict(x_grid.reshape(-1, 1))
ax.scatter(x_train.ravel(), y_train,
           c=[colors[l] for l in train_labels],
           s=22, alpha=.55, label="Training data (K-Means cluster)")
ax.scatter(x_test.ravel(), y_test, c="black", s=11, alpha=.30, label="Test data")
for label in range(N_CLUSTERS):
    mask = grid_labels == label
    order = np.argsort(x_grid[mask])
    x_part = x_grid[mask][order]
    y_part = local_models[label].predict(x_part.reshape(-1, 1))
    ax.plot(x_part, y_part, color=colors[label], lw=3,
            label=f"Local OLS: cluster {label}")
ax.plot(x_grid, global_model.predict(x_grid.reshape(-1, 1)), "--", color="#d62728", lw=2.5,
        label="Global polynomial OLS")
ax.set(title="Hybrid K-Means + Local OLS versus Global OLS", xlabel="x", ylabel="y")
ax.legend(ncol=2, fontsize=9)
fig.tight_layout()
fig.savefig(OUT_DIR / "comparison_plot.png", dpi=180)
plt.close(fig)

print("\nModel comparison on held-out test data")
print(metrics.to_string(index=False, float_format=lambda v: f"{v:.4f}"))
print("\nLocal polynomial coefficients (ordered by cluster center)")
print(pd.DataFrame(local_rows).sort_values("center_x").to_string(index=False, float_format=lambda v: f"{v:.4f}"))
print(f"\nFiles saved in: {OUT_DIR}")
```

```
if __name__ == "__main__":
    main()
```